

Analyse des Principes de Conception Logicielle

1 Analyse du respect des principes

1.1 SRP – Single Responsibility Principle

Non respecté.

La classe `GestionnaireEmploiDuTemps` gère à la fois la gestion des cours et la notification des observateurs. Elle assume plusieurs responsabilités.

1.2 OCP – Open/Closed Principle

Respecté.

Grâce au pattern Decorator, il est possible d'ajouter de nouveaux types de cours (en ligne, magistral, etc.) sans modifier le code existant.

1.3 LSP – Liskov Substitution Principle

Respecté.

Les décorateurs implémentent `ICours` et peuvent remplacer un `Cours` sans briser le comportement attendu.

1.4 ISP – Interface Segregation Principle

Respecté.

Les interfaces sont petites et ciblées (`ICours`, `Observer`, `Subject`), aucune classe n'est forcée d'implémenter des méthodes inutiles.

1.5 DIP – Dependency Inversion Principle

Non respecté.

Le `CoursBuilder` dépend directement de `Cours` au lieu de dépendre d'une abstraction. Cela crée un couplage fort.

1.6 DRY – Don't Repeat Yourself

Respecté.

Le code évite la duplication : les décorateurs réutilisent le comportement via la composition au lieu du copier-coller. Les observateurs suivent une interface unique.

1.7 KISS – Keep It Simple, Stupid

Respecté (globalement).

L'architecture reste simple : Builder, Observer, Decorator sont utilisés de manière claire sans sur-complexité.

1.8 YAGNI – You Aren't Gonna Need It

Respecté.

Aucune fonctionnalité ajoutée “au cas où”. Le code reste minimal, juste ce qui est nécessaire pour passer les tests.

1.9 COI – Composition Over Inheritance

Partiellement non respecté.

Le Decorator utilise bien la composition, mais la hiérarchie `CoursDecorator` introduit un héritage parfois inutile.

1.10 POLA – Principle Of Least Astonishment

Respecté.

Les comportements du code sont intuitifs : les décorateurs ajoutent un suffixe, l'observer notifie, le builder construit un objet. Rien de surprenant pour un développeur.